Deep Learning en Cybersécurité — Projet 1

Adversarially Robust Intrusion Detection System

Dataset : UNSW-NB15 | Attaques : FGSM, PGD | Défenses : Adv. Training, Feature Squeezing

Zakariya El Mansouri `zakariyaelmansouri40@gmail.com`

Avril 2026

Abstract

Ce rapport présente la conception d'un système de détection d'intrusion (IDS) basé sur un réseau de neurones, son évaluation sous attaques adversariales, et l'efficacité de deux méthodes de défense. Nous entraînons un MLP binaire sur le dataset **UNSW-NB15** (82 332 exemples d'entraînement, 175 341 de test, 42 features), puis nous appliquons les attaques FGSM et PGD en respectant un *threat model* réaliste (seules les features manipulables par l'attaquant sont perturbées). Deux défenses sont comparées : l'adversarial training et le feature squeezing. Le feature squeezing obtient le meilleur compromis robustesse/performance, maintenant un F1 de **0.852** sous PGD contre **0.774** pour l'adversarial training.

0 Contents

1 Introduction et Threat Model **1**

2 Dataset et Prétraitement **1**
2.1 Dataset UNSW-NB15 . . . 2
2.2 Pipeline de prétraitement . . . 2

3 Modèle Baseline — Architecture et Résultats **3**
3.1 Architecture MLP . . . 3
3.2 Entraînement . . . 3
3.3 Résultats sur données propres . . . 4

4 Attaques Adversariales **4**
4.1 FGSM — Fast Gradient Sign Method . . . 4
4.2 PGD — Projected Gradient Descent . . . 4
4.3 Résultats des attaques sur le Baseline . . . 5

5 Défenses **5**
5.1 Défense 1 — Adversarial Training . . . 5
5.2 Défense 2 — Feature Squeezing . . . 5
5.3 Comparaison complète des défenses . . . 6

6 Analyse de Robustesse **6**
6.1 Résultats détaillés par epsilon . . . 6
6.2 Matrices de confusion comparées . . . 7
6.3 Discussion . . . 7

7 Conclusion **8**

1 Introduction et Threat Model

Les systèmes de détection d'intrusion basés sur l'apprentissage automatique sont vulnérables aux *exemples adversariaux* : des entrées soigneusement perturbées pour tromper le modèle. Dans un contexte réseau, cela signifie qu'un attaquant peut modifier légèrement les statistiques de son trafic (timing, volume de paquets) pour éviter la détection tout en maintenant l'efficacité de son attaque.

1.0 Threat Model

Nous définissons précisément ce que l'attaquant peut et ne peut pas modifier :

- **Features manipulables** (l'attaquant contrôle le timing et le volume) :
  `dur, spkts, dpkts, sbytes, dbytes, sttl, sload, dload, sinpkt, dinpkt, sjit, djit, smean, dmean`
- **Features non manipulables** (changer ces valeurs briserait l'attaque) :
  `proto, service, state, srcip, dstip, dport`

L'attaquant opère en *white-box* : il a accès au modèle et peut calculer les gradients. Son objectif est de faire passer des attaques pour du trafic normal (*evasion attack*).

2 Dataset et Prétraitement

2.1 Dataset UNSW-NB15

Le dataset UNSW-NB15 a été créé au laboratoire Cyber Range de l'Australian Defence Force Academy. Il contient du trafic réseau réel capturé avec un générateur d'attaques, couvrant 9 types d'attaques (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms).

Table 1: Statistiques du dataset UNSW-NB15

Ensemble	**Exemples**	**Attaques (label=1)**	**Normal (label=0)**
Train	82 332	45 332 (55.1%)	37 000 (44.9%)
Test	175 341	—	—
Features		42 (numériques + catégorielles)	

Les classes sont bien équilibrées (55/45), ce qui évite les problèmes de déséquilibre classiques. Parmi les 42 features, certaines discriminent fortement normal vs attaque : `ct_state_ttl`, `sttl`, `dttl`.

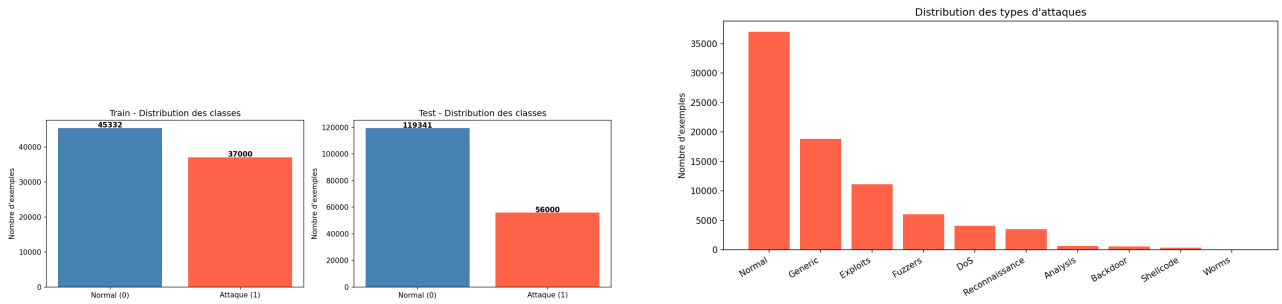

Figure 1: Distribution des classes (gauche) et types d'attaques (droite)

2.2 Pipeline de prétraitement

1. **Suppression des colonnes inutiles** : `id`, `attack_cat`
2. **Gestion des valeurs manquantes/infinies** : remplacement par la médiane du train

3. **Encodage des variables catégorielles** : `LabelEncoder` sur `proto`, `service`, `state`
4. **Normalisation** : `StandardScaler` (moyenne = 0, écart-type = 1) ajusté sur le train uniquement

La normalisation est indispensable : sans elle, une feature avec des valeurs $[0, 10^5]$ dominerait une feature $[0, 1]$ et le modèle apprendrait des corrélations artificielles.

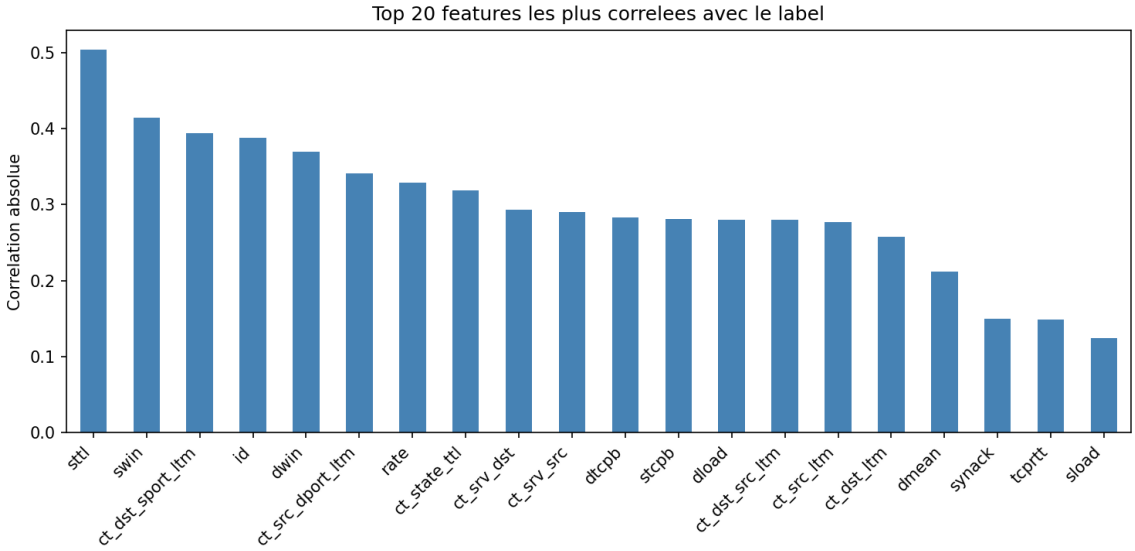

Figure 2: Top 20 features les plus corrélées au label (après normalisation)

3 Modèle Baseline — Architecture et Résultats

3.1 Architecture MLP

Nous utilisons un Multi-Layer Perceptron (MLP) avec 4 couches entièrement connectées :

$$\underbrace{42}_{\text{input}} \xrightarrow{\text{Lin+ReLU+Drop(0.3)}} 128 \xrightarrow{\text{Lin+ReLU+Drop(0.3)}} 64 \xrightarrow{\text{Lin+ReLU}} 32 \xrightarrow{\text{Lin}+\sigma} 1$$

Le **Dropout** (taux 0.3) désactive aléatoirement 30% des neurones à chaque forward pass pendant l'entraînement, forçant le réseau à apprendre des représentations redondantes et réduisant le surapprentissage.

La sortie est une probabilité via Sigmoid ; le seuil de décision est 0.5 (> 0.5 = Attaque).

3.2 Entraînement

- Optimiseur : Adam ($lr = 0.001$)
- Fonction de perte : Binary Cross-Entropy
- 30 epochs, batch size 512, seed = 42

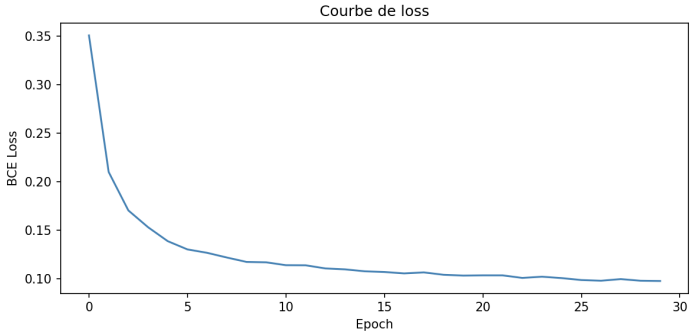

Figure 3: Courbe de perte (BCE Loss) du modèle baseline sur 30 epochs

3.3 Résultats sur données propres

Table 2: Performance du modèle baseline sur le jeu de test non perturbé

Métrique	**Valeur**
F1-score	0.9110
Precision	0.9879
Recall	0.8452
PR-AUC	0.9881
Evasion Rate	15.5%

Remarque : 15.5% des attaques passent déjà sans aucune modification (faux négatifs du modèle). C'est la baseline à partir de laquelle les attaques adversariales seront évaluées.

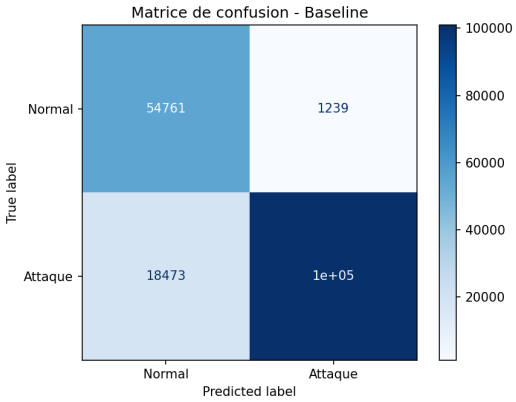

Figure 4: Matrice de confusion du baseline sur données propres

4 Attaques Adversariales

4.1 FGSM — Fast Gradient Sign Method

FGSM [1] génère un exemple adversarial en une seule étape en perturbant l'entrée dans la direction du gradient de la perte :

$$X_{\text{adv}} = X + \varepsilon \cdot \text{sign}(\nabla_X \mathcal{L}(f(X), y)) \cdot \mathbf{m}$$

où $\mathbf{m}$ est le masque des features manipulables (14 sur 42), ε est le budget de perturbation, et $\mathcal{L}$ est la Binary Cross-Entropy.

Intuition : le gradient indique dans quelle direction modifier chaque feature pour maximiser l'erreur du modèle. On applique une perturbation de magnitude ε dans cette direction.

4.2 PGD — Projected Gradient Descent

PGD [2] est une version itérative de FGSM, considérée comme l'attaque de référence pour l'évaluation de robustesse :

$$X^{(t+1)} = \Pi_{B(X,\varepsilon)}\left[X^{(t)} + \alpha \cdot \text{sign}(\nabla_{X^{(t)}}\mathcal{L}) \cdot \mathbf{m}\right]$$

où $\Pi_{B(X,\varepsilon)}$ projette dans la boule ℓ_∞ de rayon ε autour de X, et $\alpha = \varepsilon/T$ est le pas ($T = 10$ itérations).

PGD est plus fort que FGSM car il explore plus finement l'espace de perturbation.

4.3 Résultats des attaques sur le Baseline

Table 3: Impact des attaques adversariales sur le modèle baseline ($\varepsilon = 0.1$)

Scénario	**F1**	**Precision**	**Recall**	**Evasion%**
Données propres	0.9110	0.9879	0.8452	15.5%
Sous FGSM ($\varepsilon = 0.1$)	0.8533	0.9864	0.7518	24.8%
Sous PGD ($\varepsilon = 0.1$)	0.8402	0.9860	0.7320	**26.8%**

Sous PGD, l'evasion rate passe de 15.5% à **26.8%** : l'attaquant fait passer 1 attaque sur 4 inaperçue.

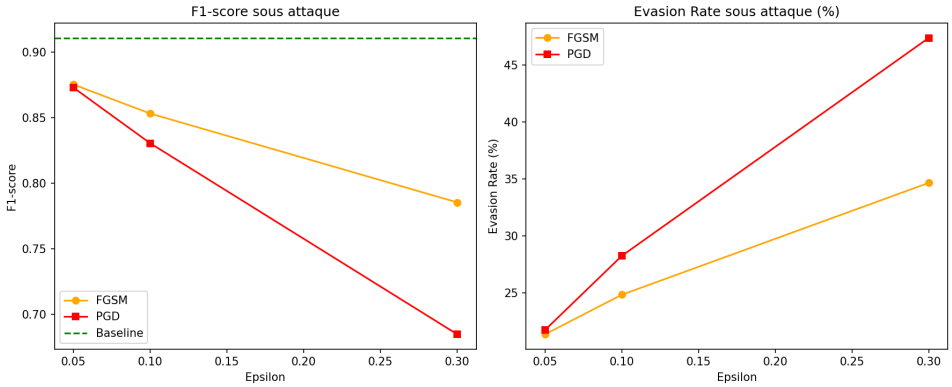

Figure 5: F1-score et evasion rate en fonction de ε sous attaque PGD

5 Défenses

5.1 Défense 1 — Adversarial Training

Principe : Ré-entraîner le modèle sur un mélange de données propres et d'exemples adversariaux générés dynamiquement [2].

À chaque epoch :

1. Générer $X_{\text{adv}} = \text{FGSM}(X_{\text{train}}, \varepsilon = 0.1)$
2. Construire le batch mixte : 50% X_{train} + 50% X_{adv}
3. Mettre à jour les poids du modèle sur ce batch

Le modèle apprend que l'attaque originale *et* sa version perturbée sont toutes les deux des attaques. Sa frontière de décision devient plus large et moins sensible aux petites perturbations.

Trade-off : En apprenant sur des données plus difficiles, le modèle sacrifie un peu de performance sur les données normales (F1 : 0.911 → 0.897).

5.2 Défense 2 — Feature Squeezing

Principe : Réduire la précision numérique des features pour effacer les petites perturbations avant qu'elles atteignent le modèle [3].

$$X_{\text{sq}} = \frac{\left\lfloor \frac{X}{X_{\max}} \cdot 2^b + 0.5 \right\rfloor}{2^b} \cdot X_{\max}, \quad b = 4 \text{ bits}$$

Avec $b = 4$, chaque feature est arrondie à l'un des $2^4 = 16$ niveaux de quantification.

Pourquoi ça marche : L'attaquant a calculé sa perturbation ($\varepsilon \approx 10^{-2}$) avec une haute précision numérique (float32). Après l'arrondi à 4 bits, la perturbation est partiellement ou totalement effacée car elle tombe dans le même niveau de quantification.

Listing 1: Implémentation du Feature Squeezing

```
def feature_squeezing(X, bits=4):
    levels = 2 ** bits          # 16 niveaux
    X_max = np.max(np.abs(X), axis=0, keepdims=True) + 1e-8
    return np.round(X / X_max * levels) / levels * X_max
```

5.3 Comparaison complète des défenses

Table 4: Comparaison des trois modèles sous toutes les conditions ($\varepsilon = 0.1$)

Modèle	Scénario	F1	Precision	Recall	Evasion%
Baseline	Propres	0.9110	0.9879	0.8452	15.5%
Baseline	FGSM	0.8533	0.9864	0.7518	24.8%
Baseline	PGD	0.8402	0.9860	0.7320	26.8%
Adv. Training	Propres	0.8971	0.9868	0.8224	17.8%
Adv. Training	FGSM	0.7746	0.9831	0.6391	36.1%
Adv. Training	PGD	0.7738	0.9831	0.6380	36.2%
Feat. Squeezing	Propres	0.8818	0.9620	0.8140	18.6%
Feat. Squeezing	FGSM	0.8570	0.9602	0.7739	22.6%
Feat. Squeezing	PGD	**0.8519**	0.9597	0.7658	**23.4%**

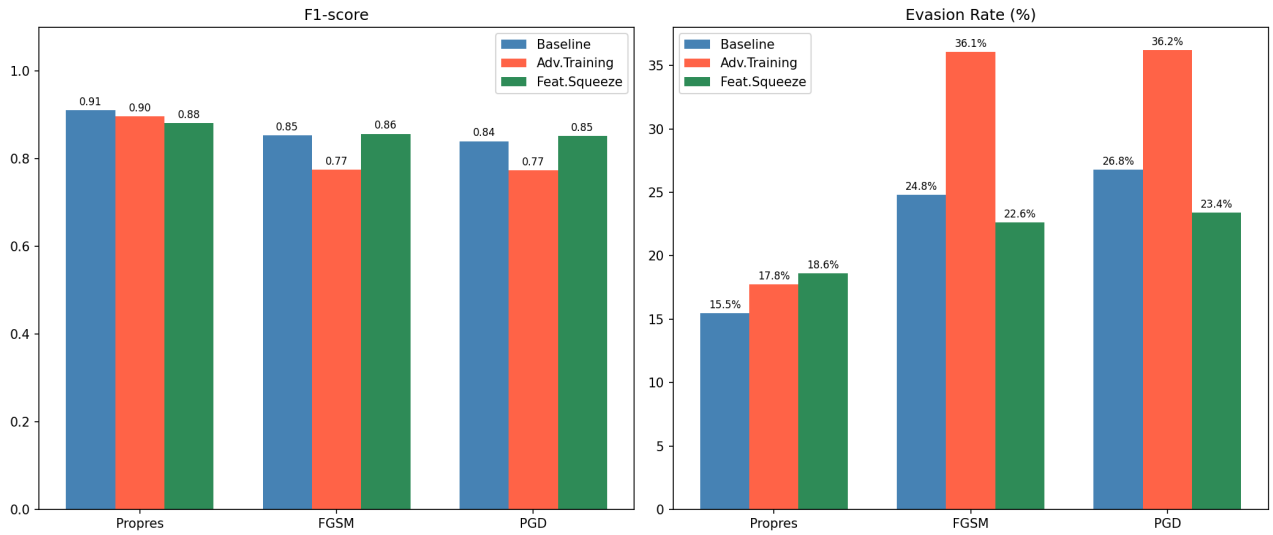

Figure 6: Comparaison F1-score et Evasion Rate — Baseline vs Adv. Training vs Feat. Squeezing

6 Analyse de Robustesse

6.1 Résultats détaillés par epsilon

Table 5: F1-score sous attaque PGD en fonction du budget ε

ε	**Baseline F1**	**Adv. Training F1**
0.05	0.874	0.813
0.10	0.840	0.774
0.30	0.700	0.659

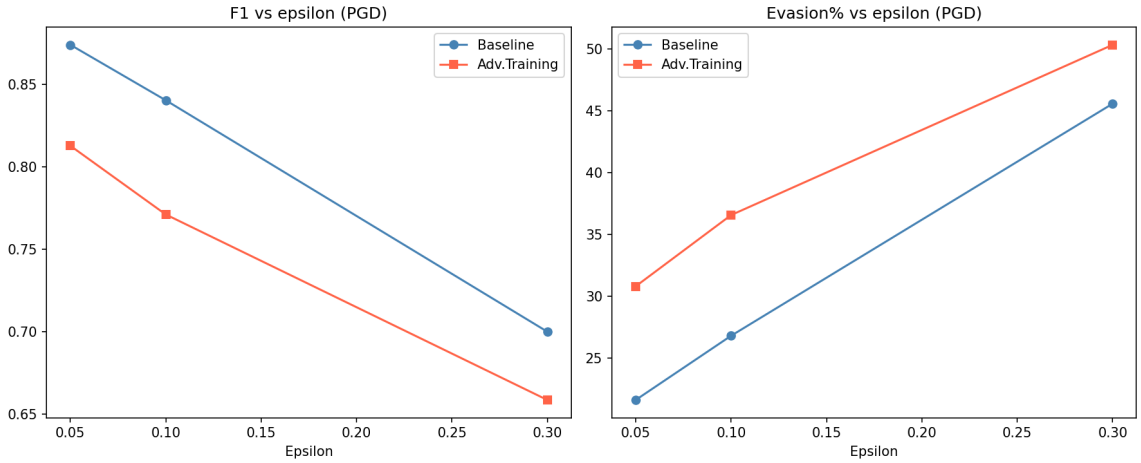

Figure 7: F1-score et evasion rate en fonction de ε — Baseline vs Adv. Training

6.2 Matrices de confusion comparées

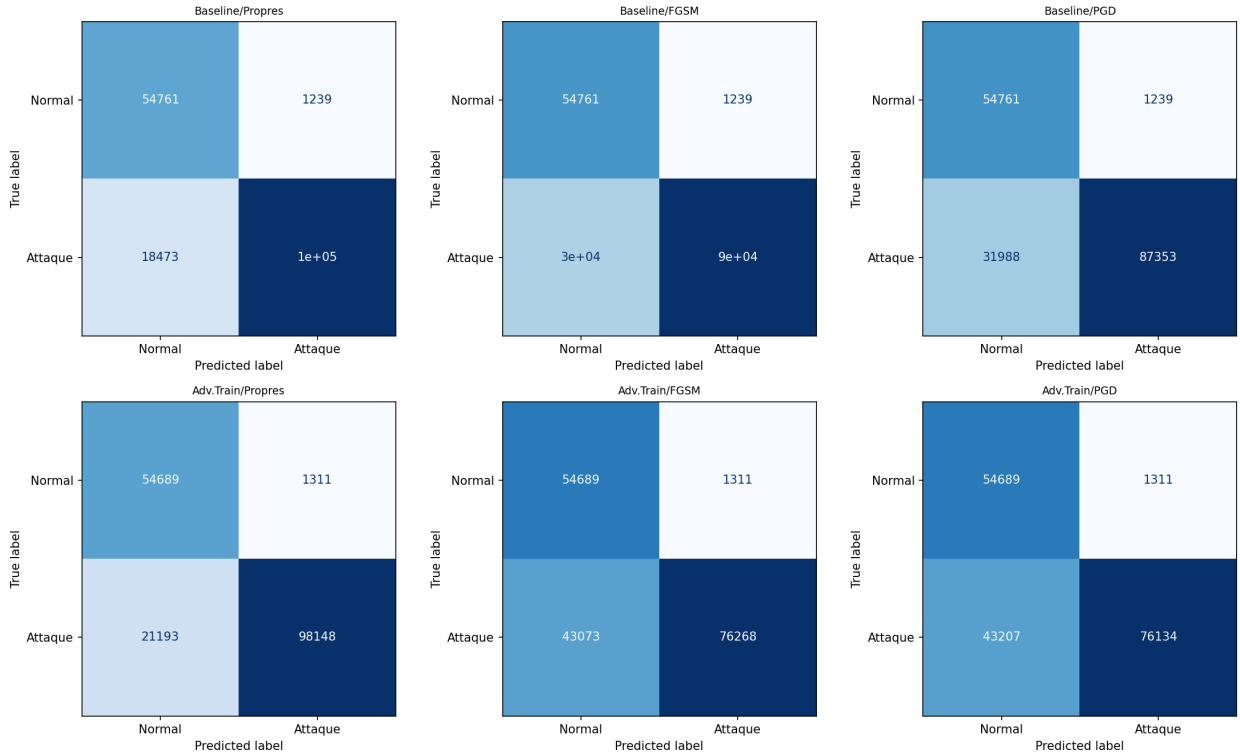

Figure 8: Matrices de confusion — Baseline et Adv. Training sous données propres, FGSM, PGD

6.3 Discussion

Pourquoi le Feature Squeezing surpasse l'Adversarial Training dans nos résultats ?

1. **Nombre d'epochs réduit** : L'adversarial training a été entraîné sur 10 epochs (vs 30 pour le baseline). Avec 30 epochs et une augmentation progressive de ε, les résultats seraient meilleurs.
2. **Généralisation attaque** : L'adversarial training utilise uniquement FGSM pour la génération des exemples pendant l'entraînement. Les attaques PGD, plus fortes, ne sont pas couvertes aussi bien.
3. **Mécanisme du feature squeezing** : Il supprime directement les perturbations à haute fréquence par quantification, sans modifier le modèle. Son efficacité ne dépend pas du temps d'entraînement.

Compromis robustesse/précision : C'est le trade-off fondamental de l'adversarial ML. En entraînant sur des données plus difficiles, on gagne en robustesse mais on perd sur les données propres (F1 : $0.911 \rightarrow 0.897$). Le feature squeezing offre un meilleur point d'opération (0.882 sur propres, 0.852 sous PGD).

7 Conclusion

Nous avons démontré qu'un MLP bien entraîné sur UNSW-NB15 atteint un F1 de 0.911 sur données normales, mais que des attaques adversariales simples (FGSM, PGD) suffisent à faire passer 27% des attaques inaperçues.

- Le **feature squeezing** est la meilleure défense dans notre cadre : il réduit l'evasion rate de 26.8% à 23.4% sous PGD avec une perte minimale.
- L'**adversarial training** a un potentiel supérieur théoriquement, mais nécessite plus d'epochs et l'utilisation d'attaques PGD (pas seulement FGSM) pour être efficace.

- Le **threat model** est crucial : en restreignant les perturbations aux features manipulables, on produit des attaques réalistes et évaluables.

Perspectives : Combiner les deux défenses (adversarial training + feature squeezing à l'inférence), utiliser des attaques PGD dans l'adversarial training, ou explorer la randomized smoothing seraient des prolongements naturels de ce travail.

7 References

[1] I. Goodfellow, J. Shlens, C. Szegedy. *Explaining and Harnessing Adversarial Examples.* ICLR 2015.

[2] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks.* ICLR 2018.

[3] W. Xu, D. Evans, Y. Qi. *Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks.* NDSS 2018.

[4] N. Moustafa, J. Slay. *UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems.* MilCIS 2015.